



SISTEMAS OPERATIVOS - SOLEMNE 01 PRÁCTICO PARTE 1 FORMA B

Consideraciones

La parte práctica de la prueba solemne 02, se dividirá en 3 partes, dos de ellas se desarrollarán en el laboratorio durante la hora de clases, y la tercera se realizará fuera del horario de clases, las notas en promedio de las tres partes equivalen al 50% de la nota de solemne 01.

Esta evaluación se desarrollará en equipos de 4 a 5 estudiantes, no se admiten grupos con menos o más estudiantes, estos se conformarán durante la primera hora de laboratorio, y se mantendrán para las tres partes del laboratorio.

La evaluación no se puede realizar de forma individual, por ende, todo aquel estudiante que quede sin equipo o grupo que no complete el mínimo establecido, será asignado, por parte del docente a un equipo, considere que:

- La sección de los miércoles y viernes tendrá un máximo de 6 grupos
- La sección de los viernes tendrá un máximo de 5 grupos.

La asistencia es obligatoria, la oficialización de los equipos se hará al inicio de la primera hora de laboratorio, con los estudiantes presentes, no se podrá asignar una persona a quienes no estén presentes, por ende, todo aquel que llegue atrasado, (pasado 10 minutos de la clase) serán asignados por el docente a un equipo.

Si un estudiante no está presente durante cualquiera de las horas de laboratorio, recibirá un 1.0 en la parte que se debe desarrollar en el horario respectivo.

Instrucciones

- Cargue los entregables solicitados en el recurso BlackBoard dispuesto para esta parte.
- No se admite el uso de IA, páginas web o cualquier material externo al material del curso, el uso de cualquiera de estos elementos, por cualquier estudiante del equipo, hará que el grupo completo obtenga un 1.0.
- Se permite el uso del material del curso, incluso los entregables del último laboratorio, para su uso debe informar al docente quien corrobora la apertura de esto recursos.
- No se admiten, entregas pasado el horario exhibido en el recurso de BlackBoard, Todo entregable posterior se evaluará con un 1.0.

Contexto

Una red de estaciones ambientales registra mediciones periódicas de temperatura, humedad, material particulado y nivel de ruido. Cada estación genera un archivo estructurado durante una ventana horaria determinada. El centro de monitoreo necesita procesar los archivos, detectar alertas ambientales y generar un resumen por estación. Dado que las estaciones transmiten múltiples conjuntos de datos, se requiere construir una versión secuencial y una versión concurrente del sistema de monitoreo, desarrolle dos versiones de un programa denominado Monitor de Estaciones Ambientales:

- Una versión secuencial.
- Una versión concurrente.

El programa debe leer archivos de ventas desde la carpeta entrada/, validar sus mediciones y producir reportes por estado ambiental por estación, para desarrollarlo, puede usar cualquiera de estos lenguajes, Python, Java o C. El lenguaje elegido deberá mantenerse obligatoriamente en las Partes 2 y 3 de la evaluación.

Estructura de proyecto

```
solemne_so_equipoXX/  
├── entrada/  
│   ├── estacion_STG01_20260902.jsonl  
│   ├── estacion_STG02_20260902.jsonl  
│   └── ...  
├── salida/  
│   ├── informe_STG01_20260902.txt  
│   ├── informe_STG02_20260902.txt  
│   └── resumen_ambiental.txt  
├── alertas/  
│   └── alertas_detectadas.log  
├── src/  
│   ├── secuencial.[py|java|c]  
│   └── concurrente.[py|java|c]  
├── evidencias/  
│   └── comparacion_parte1.txt  
└── README.md
```

Formato obligatorio de archivos de entrada

Cada archivo de entrada debe tener extensión .jsonl y representar las mediciones de una única estación durante un período determinado.

Un archivo JSONL (JSON Lines) es un archivo de texto donde cada línea contiene un valor JSON completo e independiente, normalmente un objeto. Se usa para guardar muchos registros estructurados y procesarlos uno a uno, sin tener que cargar todo el archivo en memoria.

Ejemplo

Un archivo llamado usuarios.jsonl podría contener:

```
{"id": 1, "nombre": "Ana", "pais": "Chile"}  
{"id": 2, "nombre": "Luis", "pais": "Argentina"}  
{"id": 3, "nombre": "Carla", "pais": "Perú"}
```

El nombre debe respetar el formato: estacion_CODIGO_AAAAMMDD.jsonl

Ejemplos:

- estacion_STG01_20260902.jsonl
- estacion_VAL02_20260902.jsonl
- estacion_CON03_20260902.jsonl

Cada línea del archivo corresponde a un objeto JSON independiente. No debe existir cabecera, ejemplo:

```
{"timestamp": "2026-09-02T08:00:00", "estacion": "STG01", "temperatura": 18.4, "humedad": 62.1, "pm25": 12.7, "ruido_db": 58.0}  
{"timestamp": "2026-09-02T08:10:00", "estacion": "STG01", "temperatura": 19.1, "humedad": 60.5, "pm25": 17.3, "ruido_db": 61.4}  
{"timestamp": "2026-09-02T08:20:00", "estacion": "STG01", "temperatura": 20.2, "humedad": 59.8, "pm25": 38.1, "ruido_db": 76.0}
```

Reglas de validación

Cada registro válido debe cumplir estas condiciones:

Campo	Regla
timestamp	Debe usar formato AAAA-MM-DDTHH:MM:SS
estacion	Debe coincidir con el código definido en el nombre del archivo
temperatura	Número entre -20.0 y 60.0
humedad	Número entre 0.0 y 100.0
pm25	Número mayor o igual a 0.0
ruido_db	Número entre 0.0 y 140.0

Cada archivo debe contener como mínimo:

- 15 mediciones.
- Tres horarios distintos.
- Al menos una medición que active una alerta.
- Al menos una línea inválida, ya sea por JSON mal formado, atributo ausente, estación inconsistente o valor fuera de rango.

El conjunto de entrada debe incluir como mínimo 20 archivos .jsonl.

Reglas de alerta

Una medición válida debe registrarse como alerta si cumple, al menos, una condición:

Indicador	Alerta si
Temperatura	Mayor o igual a 35.0
Humedad	Menor o igual a 20.0
PM2.5	Mayor o igual a 35.0
Ruido	Mayor o igual a 75.0 dB

Una misma medición puede activar más de una alerta.

Requisitos

Ambas versiones deben:

1. Detectar y leer todos los archivos .jsonl dentro de entrada/.
2. Procesar cada línea como una medición independiente.
3. Validar estructura JSON, atributos obligatorios, tipos de datos, rangos y consistencia del código de estación.
4. Para cada archivo, calcular utilizando solo mediciones válidas:
 - Cantidad total de líneas leídas.
 - Cantidad de mediciones válidas.
 - Cantidad de mediciones inválidas.
 - Temperatura promedio.
 - Humedad promedio.
 - PM2.5 máximo registrado.
 - Ruido máximo registrado.
 - Cantidad total de alertas.
 - Hora de la primera alerta, si existe.
5. Registrar cada alerta en: "alertas/alertas_detectadas.log"



Cada registro de alerta debe incluir:

archivo;estacion;timestamp;indicador;valor;umbral

6. Generar un informe individual por archivo en salida/.
7. Generar un resumen consolidado llamado: "salida/resumen_ambiental.txt"
8. Evitar que los informes de una estación sobrescriban los de otra estación.
9. Procesar correctamente, como mínimo, 20 archivos de entrada.
10. Medir el tiempo total de ejecución de cada versión.

Contenido mínimo de cada reporte individual

INFORME DE ESTACIÓN AMBIENTAL

```
Archivo procesado: estacion_STG01_20260902.jsonl
Estación: STG01
Fecha del archivo: 2026-09-02
Líneas leídas: 24
Mediciones válidas: 22
Mediciones inválidas: 2
Temperatura promedio: 21.45 °C
Humedad promedio: 57.20 %
PM2.5 máximo: 42.30 µg/m³
Ruido máximo: 78.50 dB
Alertas detectadas: 4
Primera alerta: 2026-09-02T08:20:00
```

Contenido mínimo del reporte consolidado

RESUMEN CONSOLIDADO DE MONITOREO AMBIENTAL

```
Versión ejecutada: secuencial | concurrente
Archivos procesados: 20
Líneas leídas: 480
Mediciones válidas: 440
Mediciones inválidas: 40
Temperatura promedio global: 20.87 °C
PM2.5 máximo global: 58.70 µg/m³
Ruido máximo global: 82.10 dB
Alertas totales: 63
Estación con mayor cantidad de alertas: STG04
Tiempo total de ejecución: 0.672 segundos
Cantidad de trabajadores: 1 | 3 o más
```

Requisitos de versión secuencial

- Procesar completamente cada archivo antes de comenzar el siguiente.
- No utilizar hilos, procesos hijos, pools ni bibliotecas de concurrencia.
- Generar resultados correctos y reproducibles para el mismo conjunto de mediciones.

Requisitos de versión concurrente

1. Implementar al menos tres unidades de trabajo concurrentes:
 - Python: `threading.Thread`.
 - Java: `Thread`, `Runnable` o `ExecutorService`.
 - C: `pthread_create()` y `pthread_join()`.
2. Distribuir los archivos mediante una estructura de trabajo compartida:
 - Python: `queue.Queue`.
 - Java: `BlockingQueue`.
 - C: cola manual protegida mediante mutex y mecanismos de espera apropiados.
3. Proteger mediante exclusión mutua los recursos compartidos:
 - Total de líneas, mediciones válidas e inválidas.
 - Sumas utilizadas para promedios globales.
 - Máximos globales de PM2.5 y ruido.
 - Contador global de alertas.
 - Identificación de estación con mayor cantidad de alertas.
 - Archivo `alertas/alertas_detectadas.log`, si se escribe desde varios trabajadores.
4. Esperar la finalización ordenada de todos los trabajadores antes de generar el resumen consolidado.
5. Garantizar que los resultados concurrentes sean iguales a los obtenidos por la versión secuencial para el mismo conjunto de archivos.

Comparación obligatoria

El archivo `evidencias/comparacion_parte1.txt` debe incluir:

Métrica	Secuencial	Concurrente
Archivos procesados		
Mediciones válidas		
Mediciones inválidas		
Alertas detectadas		
PM2.5 máximo global		



Métrica	Secuencial	Concurrente
Ruido máximo global		
Tiempo de ejecución		
Cantidad de trabajadores	1	3 o más

A continuación, una breve explicación de que contempla cada dimensión de la tabla anterior

Métrica	Qué registrar en “Secuencial”	Qué registrar en “Concurrente”
Archivos procesados	Cantidad de archivos .jsonl que el programa leyó y procesó completamente, uno a uno	Cantidad de archivos.jsonl procesados por el conjunto de trabajadores; debe ser igual a la versión secuencial
Mediciones válidas	Suma total de las líneas que pudieron interpretarse como JSON válido y que cumplen formato, campos obligatorios, tipo de datos, rangos y coincidencia de estación	Suma total de mediciones válidas procesadas por todos los trabajadores; debe ser exactamente igual al valor secuencial
Mediciones inválidas	Suma total de líneas descartadas por JSON mal formado, campos faltantes, estación inconsistente o valores fuera de los rangos definidos	Suma total de líneas inválidas detectadas por todos los trabajadores; debe ser igual a la versión secuencial
Alertas detectadas	Cantidad total de alertas generadas considerando las mediciones válidas que superan uno o más umbrales de temperatura, humedad, PM2.5 o ruido	Cantidad total de alertas detectadas por todos los trabajadores; debe coincidir con el total de la versión secuencial
PM2.5 máximo global	Mayor valor de PM2.5 encontrado entre todas las mediciones válidas de todos los archivos procesados	Mayor valor de PM2.5 calculado por los trabajadores; debe ser igual al máximo obtenido secuencialmente
Ruido máximo global	Mayor valor de ruido_db encontrado entre todas las mediciones válidas de todos los archivos procesados	Mayor valor de ruido_db obtenido por los trabajadores; debe coincidir con el máximo de la versión secuencial
Tiempo de ejecución	Tiempo total desde el inicio de lectura del primer archivo hasta la generación del resumen consolidado secuencial	Tiempo total desde el inicio del procesamiento concurrente hasta que todos los trabajadores terminan y se genera el resumen consolidado



Métrica	Qué registrar en “Secuencial”	Qué registrar en “Concurrente”
Cantidad de trabajadores	Debe registrarse 1, pues un único flujo procesa un archivo completo antes de comenzar el siguiente	Debe registrarse el número real de trabajadores creados; debe ser 3 o más

Además, debe incluir una explicación de al menos 100 palabras sobre:

- Diferencia entre concurrencia y paralelismo.
- Recursos compartidos por los trabajadores.
- Sección crítica aplicada a acumuladores, máximos y alertas.
- Riesgos de escrituras simultáneas en `alertas_detectadas.log`.
- Razones por las cuales la concurrencia puede aumentar la capacidad de respuesta, pero no asegurar una aceleración proporcional.
- Efecto que podrían tener el acceso a disco, la carga de validación JSON y los costos de coordinación.

Evidencia mínima:

El archivo zip que genere y debe subir a blackboard, debe contemplar al menos los siguientes elementos:

Elemento	Evidencia requerida
Archivos de entrada	Al menos 20 archivos <code>estacion_CODIGO_AAAAMMDD.jsonl</code> , con mediciones válidas, al menos una alerta y al menos una línea inválida
Código secuencial	<code>src/secuencial.[py java c]</code>
Código concurrente	<code>src/concurrente.[py java c]</code>
Informes individuales	Un informe por estación o archivo procesado dentro de <code>salida/</code>
Resumen consolidado	<code>salida/resumen_ambiental.txt</code>
Bitácora de alertas	<code>alertas/alertas_detectadas.log</code>
Comparación	<code>evidencias/comparacion_parte1.txt</code>
Documentación	<code>README.md</code>

Rubrica

Criterio	Destacado (71% a 100%)	Adecuado (41% a 70%)	Básico (21% a 40%)	Insuficiente (0% a 20%)	Puntaje máximo
Versión secuencial	Lee correctamente todos los archivos JSONL, valida la estructura JSON, los atributos, tipos, rangos y consistencia de la estación. Calcula promedios, máximos, alertas y primera alerta; genera informes individuales correctos.	Procesa casi todos los archivos y calcula la mayoría de los indicadores; presenta errores menores en validación, cálculo o formato.	Procesa parcialmente, omite varios indicadores ambientales o presenta informes incompletos.	No entrega una versión funcional o no procesa correctamente los archivos de mediciones.	8
Versión concurrente	Implementa tres o más trabajadores, distribuye correctamente los archivos JSONL, procesa cada archivo una sola vez y finaliza ordenadamente antes de generar el resumen. No hay archivos omitidos ni mediciones duplicadas.	Implementa concurrencia funcional y procesa la mayoría de los archivos, con pequeñas omisiones de coordinación o cierre.	Utiliza hilos o tareas concurrentes, pero presenta archivos o mediciones perdidas, duplicadas o resultados parciales.	No implementa una versión concurrente funcional.	12
Recursos compartidos y sincronización	Identifica y protege correctamente contadores, sumas para	Protege los principales recursos compartidos con un	Usa sincronización incompleta, innecesaria o	No protege recursos compartidos o presenta resultados	8



Criterio	Destacado (71% a 100%)	Adecuado (41% a 70%)	Básico (21% a 40%)	Insuficiente (0% a 20%)	Puntaje máximo
	promedios globales, máximos, alertas, estación con mayor criticidad y archivo de alertas compartido. Justifica claramente las secciones críticas.	mecanismo funcional, aunque la justificación sea parcial.	incorrecta; no identifica adecuadamente máximos, promedios o archivo de alertas como recursos compartidos.	inconsistentes entre ejecuciones.	
Informes, alertas y resultados	Genera un informe por estación, resumen ambiental consolidado y bitácora de alertas trazable. Registra correctamente los umbrales activados y evita pérdida o sobrescritura de registros.	Genera la mayoría de los informes y alertas con errores menores de trazabilidad, formato o consolidación.	Informes, resumen o archivo de alertas parcial, inconsistente o difícil de relacionar con las mediciones de entrada.	No genera resultados utilizables, no registra alertas o sobrescribe información relevante.	5
Comparación y análisis	Compara ambas versiones con las mismas mediciones, registra todas las métricas y explica técnicamente concurrencia, paralelismo, recursos compartidos,	Compara métricas principales y ofrece una explicación parcialmente correcta.	Presenta datos aislados, usa cargas no equivalentes o entrega una explicación limitada.	No compara resultados o no entrega análisis técnico.	4



Criterio	Destacado (71% a 100%)	Adecuado (41% a 70%)	Básico (21% a 40%)	Insuficiente (0% a 20%)	Puntaje máximo
	sección crítica y resultados temporales.				
Código y README	Código modular, legible, ejecutable y con nombres significativos. README incluye lenguaje, instrucciones de compilación o ejecución, estructura de carpetas, formato JSONL y reglas de alerta/validación.	Código funcional y ordenado; README suficiente, con omisiones menores.	Código difícil de ejecutar, redundante o con README escaso/incompleto.	Código incompleto, no ejecutable o sin README.	3
Total					40